



Tutorial & Programming S1C: Sorting Algorithms

Section A (Programming Questions)

1. The **timeit** module can be used to time function calls. An example code to time how long **bubbleSort1** takes to execute is as shown:

```
import timeit
array=[12,2,3,6,4,9,8,1]
print(timeit.timeit(lambda: bubbleSort1(array), number=1))
```

The function **bubbleSort1** will **run once** (number=1) and the time to execute it will be displayed.

To run the function **several times**, and **take the minimum** of all the timings, the following code can be used:

```
import timeit
array=[12,2,3,6,4,9,8,1]
times = timeit.repeat(lambda: bubbleSort1([12,3,2,6,4,9,8,1]), repeat=7, number=1)
print(min(times))
```

The function **bubbleSort1** will **run 7 times** (repeat=7) and the 7 timings of executing the function will be stored in a list. The minimum of the 7 timings will then be displayed.

- (a) Using the **timeit** module, write program code to find out whether **bubbleSort1** or **bubbleSort2** sorts faster.

<pre>def bubbleSort1(arr): swapflag = True passes = 0 while swapflag: swapflag = False passes = passes + 1 for i in range(len(arr)-passes): if arr[i] > arr[i+1]: Temp = arr[i] arr[i] = arr[i+1] arr[i+1] = Temp swapflag = True return arr</pre>	<pre>def bubbleSort2(arr): swapflag = True while swapflag: swapflag = False for i in range(0, len(arr)-1): if arr[i] > arr[i+1]: Temp = arr[i] arr[i] = arr[i+1] arr[i+1] = Temp swapflag = True return arr</pre>
---	--

- (b) Using the **timeit** module, write program code to find out whether **insertionSort1** or **insertionSort2** sorts faster.

```
def insertionSort1(Arr):
    for index in range(1, len(Arr)):
        unsortedValue = Arr[index]
        current = index - 1
        while current >= 0 and unsortedValue < Arr[current]:
            Arr[current + 1] = Arr[current]
            current = current - 1
        Arr[current+1] = unsortedValue
    return Arr
```

```
def insertionSort2(Arr):
    for i in range(1, len(Arr)):
        for j in range(i, 0, -1):
            if Arr[j-1] <= Arr[j]:
                break
            Arr[j], Arr[j-1] = Arr[j-1], Arr[j]
    return Arr
```

- (c) Write program code to compare the timings of bubble sort and insertion sort, using different sizes of array as the input. Use the faster bubble sort and insertion sort which you have found out in (a) and (b). Display the results in a table with 3 columns.

Column 1: Array size of **3 to 30**

Column 2: Bubble sort timing, in suitable decimal places

Column 3: Insertion sort timing, in suitable decimal places

2. **[quicksort]** The following pseudocode is a quicksort algorithm with no helper function. The array uses a zero base indexing.

```
FUNCTION quick_sort(k: ARRAY, first, last: INTEGER)
    low ← <.....A.....>
    high ← last
    list_separator ← k[INT(first+last)/2] //integer division

    REPEAT
        WHILE k[low] < list_separator DO //Scan left
            low ← low + 1
        ENDWHILE

        WHILE k[high] > list_separator DO //Scan right
            high ← high - 1
        ENDWHILE

        IF low <= high THEN //Swapping
            <.....B.....>
            low ← low + 1 //Shift right by one element
            high ← high - 1 //Shift left by one element
        ENDIF
    UNTIL low > high

    IF first < high THEN
        quick_sort(k, first, high)
    ENDIF

    IF low < last THEN
        <.....C.....>
    ENDIF
ENDFUNCTION
```

Implement the algorithm in Python and complete the missing statements labelled **A**, **B**, and **C**.

Section B (Discussion Questions)

1. An algorithm is written in Python code:

```
def mySort(array):
    lst = []
    N = len(array)
    for passes in range(N-1):
        smallest = array[0]
        for item in array:
            if item < smallest:
                smallest = item
        lst.append(smallest)
        array.remove(smallest)
    lst.append(array[0])
    return lst
```

- Using a trace table and `array=[56, 34, 24, 50, 43]`, show how the contents of `array` are sorted into order when the algorithm is followed.
- What is the sorting method of `mySort` function?
- Is this an in-place or not in-place sorting method?
- State the best case time complexity. Explain your answer.
- State the worst case time complexity. Explain your answer.

2. **[bubble sort]** Two bubble sort algorithms are shown in Python:

```
def bubbleSort_desc1(array):
    for Pass in range(1, len(array)): #N-1 passes
        for Posn in range(len(array)-Pass):
            #no of comparison decreases by 1 after each pass
            if array[Posn] < array[Posn+1]:
                array[Posn], array[Posn+1] = array[Posn+1], array[Posn]
    return array

def bubbleSort_desc2(array):
    for Pass in range(1, len(array)): #N-1 passes
        Noswap = True
        for Posn in range(len(array)-Pass):
            #no of comparison decreases by 1 after each pass
            if array[Posn] < array[Posn+1]:
                array[Posn], array[Posn+1] = array[Posn+1], array[Posn]
                Noswap = False
        if Noswap == True:
            return array
    return array
```

- Which of these algorithms will sort faster? Explain your answer.
- State the time complexity for each of these algorithms.

3. [insertion sort] Replace labelled parts A, B, C to write the recursive insertion sort in Python.

```
def InsertionSortRec(Arr, N):
    if .....A.....
        InsertionSortRec(.....B.....)
        unsortedValue = Arr[N]
        current = N-1
        while current >= 0 and unsortedValue < Arr[current]:
            Arr[current+1] = Arr[current]
            current = current - 1
        Arr[current+1] = .....C.....
        return Arr
    return Arr
```

4. [quicksort] This quicksort algorithm is taken from page 10 of the Sorting Algo notes.

```
FUNCTION Partition(Arr, First, Last) RETURNS INTEGER
    PivotValue ← Arr[First]
    LeftMark ← First + 1
    RightMark ← Last
    Done ← FALSE

    WHILE Done = FALSE DO
        WHILE LeftMark <= RightMark AND Arr[LeftMark] <= PivotValue DO
            LeftMark ← LeftMark + 1
        ENDWHILE
        WHILE LeftMark <= RightMark AND Arr[RightMark] >= PivotValue DO
            RightMark ← RightMark - 1
        ENDWHILE
        IF LeftMark > RightMark THEN
            Done ← TRUE
        ELSE
            SWAP(Arr[LeftMark], Arr[RightMark] //swaps Arr[LeftMark], Arr[RightMark]
        ENDIF
    ENDWHILE

    SWAP(Arr[First], Arr[RightMark] //swaps Arr[First], Arr[RightMark]
    RETURN RightMark //index of the pivot element that is in its correct position
ENDFUNCTION
```

```
FUNCTION QuickSort(Array, First, Last) RETURNS ARRAY OF INTEGERS
    IF First < Last THEN
        SplitPoint ← Partition(Array, First, Last)
        QuickSort(Array, First, SplitPoint - 1)
        QuickSort(Array, SplitPoint + 1, Last)
    ENDIF
    RETURN Array
ENDFUNCTION
```

Perform a dry run on the array [9, 5, 8, 12, 3, 7, 11] until the first SplitPoint using this table heading:

SplitPoint	First	Last	PivotValue	LeftMark	RightMark	Done	Arr
------------	-------	------	------------	----------	-----------	------	-----

5. **[insertion sort]** A list contains numerical data 13, 3, 7, 11, 2. Describe how an insertion sort can be used to sort this list.
6. **[merge sort]** List A contains 3, 7, 11, 13. List B contains 1, 4, 8, 10, 16, 18, 19. Describe how a merge sort can be used to merge the two lists into one list in numerical order, smallest first.
7. **[merge sort]** Draw the trace tree diagram when merge sort is applied, showing clearly the order of the steps processes:

Index	[0]	[1]	[2]	[3]	[4]	[5]
ar	9	7	8	1	2	3

8. **[merge sort]** Top-down merge sort is a divide and conquer algorithm that can be implemented using recursive functions. The partial pseudocode for top-down implementation merge sort is given below:

```

Line 1:  PROCEDURE mergeSort(ar, low, high)
Line 2:      IF low < high THEN
Line 3:          DECLARE mid: INTEGER
Line 4:          mid=INT(low+high)/2
Line 5:          mergeSort(ar, low, mid)
Line 6:          mergeSort(ar, mid+1, high)
Line 7:          merge(ar, low, mid, high)
Line 8:      ENDIF
Line 9:  ENDPROCEDURE

```

Given that `ar = [16, 12, 18, 8, 10, 37, 6, 31]` and applied to the `mergeSort` function. State the total number of times `mergeSort` is called.

9. There are many methods of sorting a set of records into ascending order of key. What factors would you consider in deciding which of these methods is the most suitable for a particular application?
10. The merge sort algorithm has time complexity of $O(n \log n)$. For a list of 1024 items in random sequence, is this algorithm more or less efficient than a sort algorithm of time complexity $O(n^2)$? Explain your answer.

Section C (Assignment Questions)

1. **[bubble sort]** Analyse the compare and swap operations of the two Python codes. Which of these codes will sort faster? Explain.

<pre>def bubbleSort1(arr): swapflag = True passes = 0 while swapflag: swapflag = False passes = passes + 1 for i in range(len(arr)-passes): if arr[i] > arr[i+1]: Temp = arr[i] arr[i] = arr[i+1] arr[i+1] = Temp swapflag = True return arr</pre>	<pre>def bubbleSort2(arr): swapflag = True while swapflag: swapflag = False for i in range(0, len(arr)-1): if arr[i] > arr[i+1]: Temp = arr[i] arr[i] = arr[i+1] arr[i+1] = Temp swapflag = True return arr</pre>
---	--

2. **[insertion sort]** Analyse the compare and swap operations of the two Python codes. Which of these codes will sort faster? Explain.

<pre>def insertionSort1(Arr): for index in range(1, len(Arr)): unsortedValue = Arr[index] current = index - 1 while current >= 0 and unsortedValue < Arr[current]: Arr[current + 1] = Arr[current] current = current - 1 Arr[current+1] = unsortedValue return Arr</pre>	<pre>def insertionSort2(Arr): for i in range(1, len(Arr)): for j in range(i, 0, -1): if Arr[j-1] <= Arr[j]: break Arr[j], Arr[j-1] = Arr[j-1], Arr[j] return Arr</pre>
--	---

3. **[merge sort]** The merge sort algorithm given on page 11 of the notes shows two functions headers as follow:

FUNCTION **mergesort**(arr: ARRAY[1:N] of INTEGERS) RETURNS ARRAY

FUNCTION **merge**(left, right: ARRAY of INTEGERS) RETURNS ARRAY

Modify the algorithm by writing only **one function** to do the merge sort.